WVA benchmarking with llm-d-sim without flowcontroller

Setup:

- Cluster started via mak-deploy-emulated-on-kind
- The flow controller is disabled
- HPA patch.yaml:

```
spec:
  behavior:
scaleUp:
stabilizationWindowSeconds: 0
metrics:
  - type: External
external:
metric:
name: wva_desired_replicas
selector:
matchLabels:
variant_name: ms-sim-llm-d-modelservice-decode
exported_namespace: llm-d-sim
target:
type: AverageValue
averageValue: 1
```

- KV cache patch.yaml

```
spec:
  template:
spec:
```

```
containers:
    - name: vllm
env:
      - name: POD_IP
valueFrom:
fieldRef:
fieldPath: status.podIP
args:
      - --model
      - unsloth/Meta-Llama-3.1-8B
      - --port
      - "8200"
      - --served-model-name
      - unsloth/Meta-Llama-3.1-8B
      - --time-to-first-token=200
      - --inter-token-latency=20
      - --enable-kvcache
      - --kv-cache-size=1024
      - --block-size=16
      - --tokenizers-cache-dir=/tmp
```

- EPP scorer config

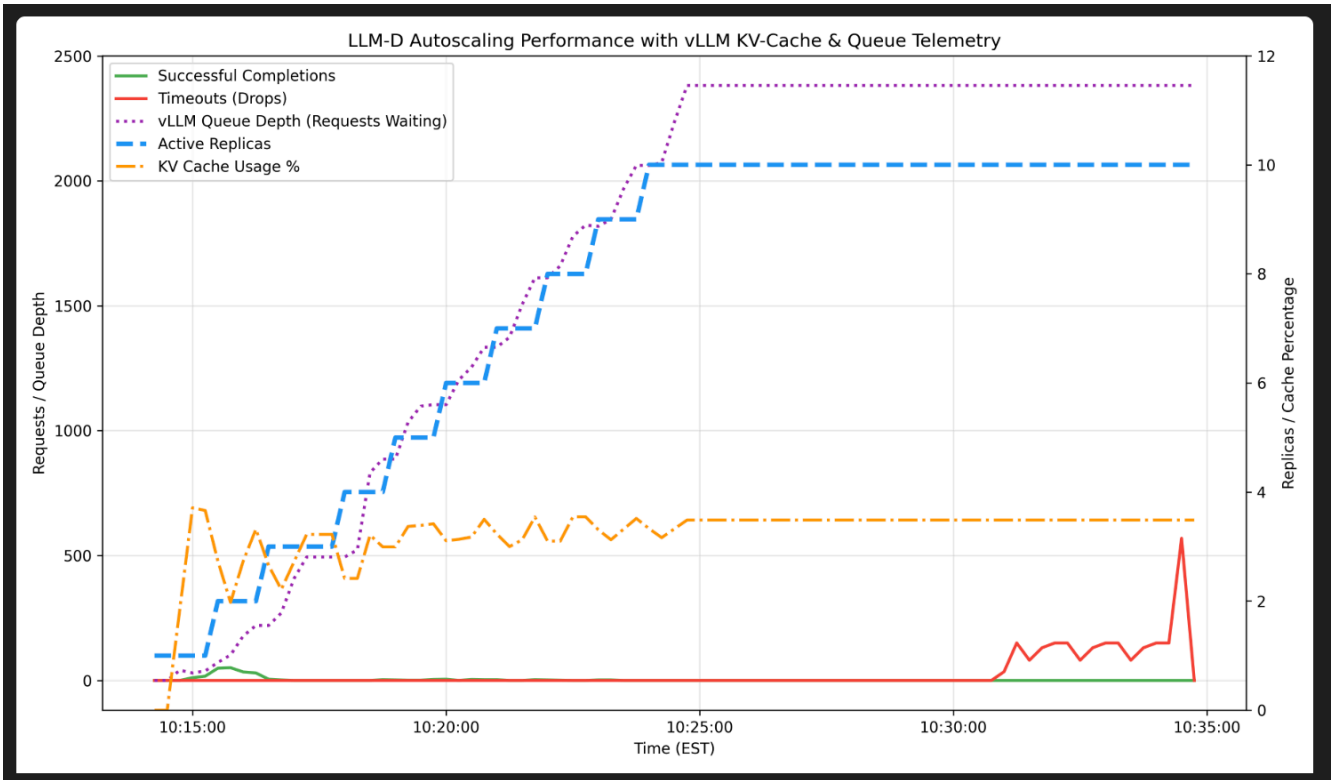
```
apiVersion: inference.networking.x-k8s.io/v1alpha1
kind: EndpointPickerConfig
plugins:
- type: queue-scorer
- type: kv-cache-utilization-scorer
- type: prefix-cache-scorer
schedulingProfiles:
- name: default
plugins:
  - pluginRef: queue-scorer
  - pluginRef: kv-cache-utilization-scorer
  - pluginRef: prefix-cache-scorer
```

- constant_traffic_20min.sh

```
#!/usr/bin/env bash


# Simulation Configuration
export LLMDBENCH_HARNESS_NAME="guidellm"
export LLMDBENCH_HARNESS_EXPERIMENT_PROFILE="constant_10rps_20min"
export LLMDBENCH_VLLM_COMMON_NAMESPACE="llm-d-sim"
export LLMDBENCH_HARNESS_NAMESPACE="llm-d-sim"


# WVA Configuration
export LLMDBENCH_WVA_ENABLED=true


# PVC Configuration
export LLMDBENCH_VLLM_COMMON_PVC_ACCESS_MODE="ReadWriteOnce"


# Harness Resources
export LLMDBENCH_HARNESS_CPU_NR=1.0
export LLMDBENCH_HARNESS_CPU_MEM=4Gi


# Explicit Endpoint Override
export LLMDBENCH_HARNESS_STACK_ENDPOINT_URL="http://infra-sim-inference-gateway-istio.llm-d-sim.svc.cluster.local:80"


# Explicit Model List to match deployment
export LLMDBENCH_DEPLOY_MODEL_LIST="unsloth/Meta-Llama-3.1-8B"
```

- Commands:

```
make deploy-wva-emulated-on-kind CREATE_CLUSTER=true DEPLOY_LLM_D=true && ./run.sh -c scenarios/constant_traffic_20min.sh
```



Run 6 Optimization Results

Total requests fired by GuideLLM: 10 requests/sec * 1200 seconds = 12,000
With the flow-controller disabled, all 12000 requests were forwarded to the vLLM servers.

The graph plots fewer dropped requests than fired because the container logs rolled over; 11765 requests were dropped, and only 235 passed. This is due to the vLLM sim server processing speed and GuideLLM timing out; the simulator is configured to output 128 tokens `--inter-token-latency=20ms` and `--time-to-first-token=200ms`. `200ms + (128 * 20ms) = ~2.76 seconds per request per pod`. With 10 replicas running at most, the theoretical maximum throughput of the entire cluster is roughly 3.6 requests per second. GuideLLM is firing 10 requests per second. The cluster is fundamentally under-provisioned and eating a massive deficit every single second. I think llmd-sim uses a strict First-In-First-Out queue without age checks; requests pile up massively in the simulator's memory. By around minute 3 of the test, the 1,800th request enters the back of the vLLM queue. It will take the cluster several minutes to finally reach it. GuideLLM has a strict `httpx.ReadTimeout` limit of 60 seconds. Since it takes llmd-sim several minutes to reach the request in its internal queue, the client always drops the TCP socket long before vLLM even starts processing the prompt. Only the very first few requests that slip into the llmd-sim queue immediately (or right as a pod initializes) are processed and returned within the 60-second lifeline.

Once the simulator has fully generated the fake LLM response, it attempts to write it back to the original TCP socket to GuideLLM. At this exact moment, the simulator's web server (the underlying Go HTTP handler) realizes the socket is dead. It receives a `Broken Pipe` socket error from the operating system, discards the generated text, and finally deletes the request from its memory. If 12,000 requests are distributed across 10 replicas (1200 per replica), and each request takes ~2.7 seconds to process, the requests depart after several minutes because, I think, llmd-sim servers are designed for uncancellable batching. The simulator wastes tens of minutes of resources generating answers for a client that does not exist.

A large number of failures at the end of the experiment is just a matter of convenience for plotting the requests that failed and are available in unrolled container logs.

Observations

- HPA stabilization window affects max replica count in experiment: Impact on max replicas on changing HPA stabilization window with llm-d-sim · Issue #777 · llm-d/llm-d-workload-variant-autoscaler
- WVA fails to scale down: WVA refusing to scale llm-d-sim modelservice back to 1 · Issue #769 · llm-d/llm-d-workload-variant-autoscaler